Thinking Ahead

Independent-thinking lessons from building ThaiBridge AI, start to finish

A personal study guide for Joshua Kay

Built from 1,075 commits (22 Feb 2026 to 28 Oct 2026) and the project journal

Prepared 3 November 2026

The skill this guide trains

My Senior Advisor's advice: **be flexible in your thinking, and be able to anticipate the next few steps.**

This guide doesn't teach you to write code. It teaches the habit of thinking that sits behind good code, and that habit carries over to farming, logistics, running a business and running a project. All the examples come from things you have already built, broken and fixed.

How to use this guide

ThaiBridge wasn't built from a perfect plan. It was built the way real projects usually are: a rough idea, a first version, then a long run of surprises. The good news is that every surprise can be turned into a lesson. Looking back over finished work and asking *"what could I have seen coming?"* is one of the fastest ways to get better at seeing what's coming next time.

Read it in three passes:

- **Pass 1 - the big picture (pages 3-5).** The six questions every project should answer, answered for ThaiBridge now that we know how things turned out.
- **Pass 2 - the journey (pages 6-7).** Each stage from November 2025 to now: what was built, what went wrong, and the thinking skill it taught.
- **Pass 3 - practise (pages 8-11).** Real incidents set out as puzzles. **Cover the answer, make your prediction, then read on.** Being wrong here is the point of the exercise.

> **The one question to carry everywhere**
>
> **"What happens next, and what else does this touch?"**
>
> Almost every surprise in this project came from something that looked finished but was quietly connected to something else: a database wiped on every deploy, a price that included tax in one place and not another, a phone layout that broke a page nobody had touched.

The four thinking moves

Move	What it means	The ThaiBridge habit
1. Zoom out	Before acting, ask what the thing is *for* and who relies on it.	"The alphabet is the way in, so it has to be free."
2. Look ahead	Run the clock forward: after this, then what? Then what?	"If progress lives in a cookie, a new phone loses it."
3. Look for the hidden link	Ask what else depends on this, or what it depends on.	"The webhook trusts metadata, not the session ID."
4. Check the source	Separate what you *know* from what you *assume*. Prove it from the real thing.	"A confident answer from the wrong source is worse than no answer."

Part 1 - The six big questions, answered

These are the six questions from the reusable prompt's "First action". At the start of a project you answer them with guesses. Answering them *now*, with hindsight, shows you what good answers look like.

1. What problem are we solving?

English speakers who are drawn to Thailand (expats, travellers, students and people interested in Theravada Buddhism) have nowhere to learn Thai *together with* the culture and the Dhamma. The March-April 2026 research confirmed it: the big language apps don't teach Thai at all, and none of them combine it with Buddhist culture.

> **Thinking skill: name the gap, then check it's real**
>
> You researched competitors and audience *before* building more. Lots of people skip this and build something nobody needs. Next time, write the problem in one sentence and find three pieces of evidence that it's real.

2. What does the finished product need to do?

- Teach the foundations: alphabet, tones, consonant classes, vocabulary and sentences, with audio for everything.
- Give learners an AI tutor that is **accurate**. A wrong answer from a tutor is worse than no answer at all.
- Respect the source material: the chanting book, Pra Kru Bob's writings, real Thai checked by native speakers.
- Earn money fairly through Free, Reader, Pro and an Instant Access Pass, while keeping the Dhamma freely given.
- Keep accounts, progress and payments safe and lasting.

3. What is the simplest appropriate technical solution?

Part	What ThaiBridge uses	Why this is the simple choice
Web app	Flask (Python)	One language for everything; small and easy to understand.
Pages	HTML templates + CSS	No heavy front-end framework needed for a content site.
Database	SQLite locally, Neon Postgres live	SQLite needs no setup; Postgres survives redeploys (see Case 3).
AI	Claude API (Sonnet 5.5) via ai_agent.py	One provider, one file, an accuracy exam guarding it.
Payments	Stripe Checkout + webhooks	Stripe handles cards, tax and receipts so you don't have to.
Hosting	Render (auto-deploy from main)	Free tier, a push to GitHub puts it live.

Part	What ThaiBridge uses	Why this is the simple choice
Safety net	Git + 970-odd automated tests	You can take risks because you can check and roll back.

4. What were the major development stages?

Looking back, the order made sense, and each stage *needed* the one before it:

#	Stage	Why it had to come here
0	Plan and cut to an MVP	Without a small target, nothing gets finished.
1	First working version + AI tutor	You need something real to test and learn from.
2	Git, tests, sign-up	A safety net before making bigger changes.
3	Market research	Check the idea is worth more time before investing it.
4	Layout, payments, going live	Real users and real money show the real problems.
5	Content tracks, access model, audio	Once it's live, make it worth paying for.
6	Chanting book + a database that lasts	Scale exposed the weak foundation.
7	Payments ledger, honest prices	Money is where mistakes hurt trust most.
8	Tutor accuracy, accounts, learning tools	Quality and safety for a growing app.

5. What problems should we anticipate?

These are the **categories** that kept coming up. When you start something new, go through this list and ask "where could this bite me?"

- **Things that look finished but aren't**: a page that loads but links nowhere, tests that pass while running against the wrong database.
- **Things that vanish**: data stored on a disk that resets, a secret key set for one terminal only, progress kept in a browser cookie.
- **Money leaks**: a public AI link spending your credits, a paid quiz left open, a price that differs from the charge.
- **Trust and accuracy**: an AI teaching wrong Thai, a summary that drops paragraphs of a sacred text.
- **Different devices**: an iPhone layout that buries the page under a sidebar.
- **Wrong source of truth**: checking an old Stripe account, trusting a code comment over the real settings.

6. What should I understand before beginning?

- **Where things live.** Most early errors were files in the wrong folder.
- **Where data lives, and how long it lasts.** Browser, server disk or database?
- **Who can see what.** What's free, what's paid, what's secret (keys in .env, never in code).
- **How you'll know it works.** Decide the test *before* you build.
- **"Done" means checked by you**, not claimed by an AI.

Part 2 - The journey, stage by stage

For each stage: **what was built**, **what went wrong or was learned**, and **the thinking skill** it trains. Try to spot the skill before you read it.

Stage 0 - Nov 2025: Planning the app

Built: A describe-then-check plan: tell the AI exactly what you want, judge what comes back, refine.

What went wrong or was learned: The plan ballooned into a huge, expensive vision. Progress only started after cutting it to a small working version.

Thinking skill: Scope control. Ask "what is the smallest version that is actually useful?"

Stage 1 - Nov-Dec 2025: First version and the AI tutor

Built: Flask backend, alphabet and vocabulary, lessons, points and levels, then a Claude-powered tutor with several modes.

What went wrong or was learned: "Template not found" was a file in the wrong place. The API key vanished because it was set for one terminal only. An AI said the chat worked when it didn't. A romanisation accent bug was so deep that a workaround was the right call.

Thinking skill: Read the error message, and **know when to stop polishing** and choose a workaround.

Stage 2 - Feb 2026: Version control and a real workflow

Built: Git and GitHub, user registration with a database, 32 tests for the points logic, a first code review.

What went wrong or was learned: Empty forms could be submitted, and a database error could crash the app. When Claude Code ran out of credits, you made two fixes by hand.

Thinking skill: Build the safety net first. With rollback available, you can take bigger risks calmly.

Stage 3 - Mar-Apr 2026: Is it worth building?

Built: Competitor, audience and pricing research, with the prompts saved in /prompts.

What went wrong or was learned: A web-scraping tool you'd named in a prompt had never actually run.

Thinking skill: Check which tools actually ran. "I asked for X" is not "X happened".

Stage 4 - Jun 2026: Layout, payments, going live

Built: A site outline and shared layout, Stripe and PayPal, real subscriptions, the first public deploy on Render, and a cost-safe AI demo.

What went wrong or was learned: A renewal-date bug turned up in the first test payment. A public AI link spends *your* credits, so you added a cheap model, a daily limit, a separate API key as a kill switch, and a hard spend cap.

Thinking skill: Ask "who pays if this goes wrong?" Layer your protections so no single one has to be perfect.

Stage 5 - Jul 2026: Making it worth paying for

Built: Monk Mode, a reworked access model, tones and consonant classes, a rebuilt alphabet, audio everywhere, an accessibility panel, Jataka tales.

What went wrong or was learned: The “broken” iPhone alphabet page was really three lines of shared CSS. A story link went to a missing page while raising no error.

Thinking skill: Find the real cause before rebuilding. The worst bugs raise no error, so make the app check itself.

Stage 6 - Aug 2026: The chanting book and a database that lasts

Built: A pipeline for 286+ chants that proves nothing was lost, and a move from SQLite to Neon Postgres.

What went wrong or was learned: The live site had been deleting every account on every deploy. Your own tests had been running against the live database.

Thinking skill: Ask “how long does this last, and where does it live?” about every piece of data.

Stage 7 - Sep 2026: Money you can trust

Built: A payments ledger, receipts on /progress, a real live-money test.

What went wrong or was learned: A £9.99 plan charged £11.99 (VAT). The first fix raised every price; the right fix was one line telling Stripe the price already includes tax. A key file was one command away from being made public.

Thinking skill: Fix the right thing. When two parts disagree, ask which one should change.

Stage 8 - Oct 2026: Quality, safety and new tools

Built: Tutor on Sonnet 5.5 with a 17-question accuracy exam, playable Tour Guide trips, password reset, sign-out-everywhere, IEP learning path, Listening and Writing tools, Translator mode with photo input.

What went wrong or was learned: The old tutor model taught wrong Thai and wrong Dhamma. The sign-out feature quietly wiped logged-out visitors' progress, caught by a review after the tests passed. Three paid quizzes were open to everyone.

Thinking skill: Measure quality, don't assume it. A review after “done” still finds things.

Part 3 - Predict before you read

Each case is a real moment from ThaiBridge. **Cover everything below "Pause and predict"**, write your answer in the margin, then uncover. The aim isn't to get it right. It's to build the reflex of *predicting* before reacting.

Case 1 - The page that wouldn't open on iPhone

The situation: The Alphabet page wouldn't work on an iPhone, so the page was rebuilt. Still broken. Rebuilt again. Still broken.

> **Pause and predict:** If rebuilding the page twice didn't help, where else could the problem be?

What actually happened: The page was never the problem. Three lines of CSS in the *shared* layout stacked a 644px sidebar on top of the content on phones. One small change (moving the main content first on small screens) fixed every page at once.

The lesson: If fixing the obvious thing doesn't work, the cause is somewhere else. Look at what the thing *shares* with others. And ask the failing device what it's actually doing before rebuilding anything.

Case 2 - The link that raised no error

The situation: Five culture stories each linked to a practice page. The page built each link by turning the story's ID into /exercise/<id>.

> **Pause and predict:** Can you spot what might go wrong with building links from IDs?

What actually happened: It worked for four stories because their IDs happened to match vocabulary categories. The fifth had no category, so its link pointed at nothing. No error, no log, a page that loaded fine. The fix: each story says where its link goes, and the app refuses to start if any link is dead.

The lesson: Anything that works "by coincidence" will break one day. Prefer stating things outright over guessing, and build checks that fail loudly.

Case 3 - The database that was deleting itself

The situation: The live site stored accounts in a SQLite file on Render's server disk. Render rebuilds the server on every deploy.

> **Pause and predict:** What happens to every account each time you push an update?

What actually happened: Every deploy wiped the file, so every account created since the last deploy was silently deleted. The fix was Neon Postgres, a database that lives separately from the server, proved by registering, redeploying and logging back in. The same session found the test suite had been running against the live database.

The lesson: For every piece of data, ask: where does it live, and what wipes it? Then prove survival by actually doing the thing that would wipe it.

Case 4 - £9.99 that charged £11.99

The situation: A live payment test showed Stripe adding 20% VAT on top of a £9.99 plan. You are not VAT registered.

> **Pause and predict:** There are two ways to make the advertised price and the charge agree. What are they, and which is fairer to customers?

What actually happened: Option one: raise the advertised price to include VAT. That was tried first and quietly raised every price by 20%. Option two: tell Stripe the price already includes tax (one line: tax_behavior: inclusive). Stripe is the merchant of record, so it pays the VAT out of the £9.99. Along the way, two wrong turns: trusting a code comment about account settings, and checking an old Stripe account from a stale .env file.

The lesson: When two things disagree, decide which one is right before changing either. And check you are looking at the *real* source.

Case 5 - The tutor that taught mistakes

The situation: The live tutor ran on a cheap, fast model to keep costs down. It sounded confident and fluent.

> **Pause and predict:** How would you find out whether a confident AI is actually correct, and what would you do about cost if you switched?

What actually happened: A side-by-side test with the same questions across three models showed the cheap one confusing mettā with muditā, inventing teaching, and getting consonant classes wrong. The tutor moved to Sonnet 5.5. A rule was set first ("it must not lose money"), so the daily limit was lowered and a monthly cap added. Now a 17-question exam runs before any change to the tutor.

The lesson: Fluent is not the same as correct. Make a test you can repeat, and set your limits (like cost) *before* you change things.

Case 6 - The safety feature that wiped progress

The situation: Changing your password now signs you out on every other device. Five tests passed.

> **Pause and predict:** A device that's been signed out still has its old login cookie. What happens on that device's next page load, and the one after?

What actually happened: The app rejected the dead cookie on every page, and each time it wiped whatever the visitor had earned since. The tests missed it; a review afterwards caught it. The fix clears the dead cookie once.

The lesson: Run the clock forward past the happy path. Ask what happens on the *second* and *third* visit, not just the first.

Case 7 - A public AI link

The situation: You want clients to try the AI tutor from your portfolio, with no login needed.

> **Pause and predict:** List three ways this could cost you money or cause trouble, and one protection for each.

What actually happened: One person could spam it (fixed with a daily message limit). A bad day could run up a big bill (a hard spend cap, auto-reload off). Abuse could break your own development work (a separate API key you can switch off on its own). Plus a cheaper model for the public demo.

The lesson: Ask "what's the worst a stranger could do with this?" Then add layers, so no single protection has to be perfect.

Part 4 - Your thinking toolkit

The anticipation checklist

Run through this **before** building or changing anything, whether it's a feature, a business offer or a job on the farm. It comes from the reusable prompt, put into plain words.

Ask yourself	ThaiBridge example
What happens straight after this is used?	After payment, the webhook must unlock the right plan.
What depends on this?	Twenty sections depend on the alphabet quiz being passed.
What could go wrong?	The quiz result is stored in a cookie and gets cleared.
What if someone does something unexpected?	An empty form, a fake Stripe event, 500 AI messages.
What if an outside service fails?	If the AI spend cap is reached, visitors see “the tutor is resting”, not an error.
Security, privacy or speed problems?	Chats kept in the learner's browser, so the server stores no message text.
What will we need next?	Accounts led to password reset, which led to sign-out-everywhere.
Could this choice cause trouble later?	SQLite on a server disk was fine until the first redeploy.

Ten habits that came out of this project

1. **Cut to the smallest useful version** before adding anything.
2. **Read the error message.** It usually names the problem.
3. **Don't trust “it works”**, from an AI or from yourself, until you've checked it.
4. **Know where every piece of data lives** and what could wipe it.
5. **Check the real source**, not a comment, a memory or an old account.
6. **Fix the cause, not the symptom.** Rebuilding the page twice didn't fix the layout.
7. **Make things fail loudly.** An app that refuses to start beats a silent dead link.
8. **Decide the test before you build.** The tutor exam came before the model switch.
9. **Protect the commitment, not just the wording.** When someone criticises how something is said, keep what it *means*.
10. **Review after “done”.** The worst bug of October was found after the tests passed.

Part 5 - Using this outside code

My Senior Advisor's point was about *real-world* activities. The same four moves work anywhere. Try them on these:

Situation	Zoom out / look ahead / hidden link / check source
Your two bee nucs arrive on 13 April 2027.	What has to be ready the day *before*? What if it rains that week? What depends on the hive stand being level? Who has actually done this locally?
A client on PeoplePerHour asks for a "quick change".	What is it really for? What else does that page touch? What will they ask for next? Is it in the agreed scope?
Loading a delivery lorry.	What gets unloaded first? What's fragile or heavy? What happens if one drop-off is cancelled?
Launching a new SMOALD offer.	Who pays if it goes wrong? What does the customer do the minute after buying? What does the price page say compared with the checkout?

A five-minute daily practice

- Pick one thing you'll do today.
- Write down **three things that happen after it**, then three after each of those.
- Circle the one that would hurt most if it went wrong. Add one protection.
- That evening, check: did anything happen that you *could* have predicted?

Working with Claude Code this way

You don't need to start a project from scratch to use the reusable prompt. For ThaiBridge, try opening a session with one of these:

- "Before you build this, ask me to predict what could go wrong, then tell me what I missed."
- "Give me two ways to do this, the trade-off between them, and your recommendation."
- "After this stage, give me the short review: built / how it works / risks / what I learned / next."

Reflection

Answer in your own words. Coming back to these in a month is the best way to see how far your thinking has moved.

Which case (1-7) did I predict correctly? What helped me see it?

Which case surprised me most? What question would have caught it?

What is one thing in my current work that "looks finished" but that I haven't proved?

Where does the most important data in my life or business live, and what could wipe it?